

PhonoScript GUI

Desktop Application Manual

Optimality Theory · Harmonic Grammar · Maximum Entropy

Serial evaluation · Second-Order Tableaux · Q-Calculus

PhonoScript language

Alexandre Menezes Barroso

alexandrebarroso.com

Version 1.2.0 · 2026

MIT License

Contents

1	Purpose and scope	1
2	Installation and first project	1
3	Workspace and project organization	2
4	Tableau editing	2
5	Evaluator mathematics	3
6	Second-Order Tableau and Q-Calculus	4
7	PhonoScript	4
8	Publication export	5
9	Keyboard and preferences	6
10	Validation and limits	7
11	Security and contribution discipline	8
12	License and attribution	8

1 Purpose and scope

PhonoScript GUI is a native environment for building, calculating, comparing, and publishing constraint-based phonological analyses. It places established first-order evaluators and the dissertation's typed comparison calculus in one project format. Source and target answers are always calculated independently; a transport aligns typed answers but never manufactures a target result.

Project PhonoScript is a two-component workspace. The standalone PhonoScript crate owns the language, document model, evaluator mathematics, comparison semantics, Q-Calculus, and export scene. PhonoScript GUI depends on that crate and adds the graphical project environment. The dependency is one-way: the language and engine run without the desktop application. This prevents GUI behavior from becoming a second implementation of the mathematics.

A `.ottab` schema-version-4 project may contain many tableaux with different OT, HG, or MaxEnt evaluators. A PhonoScript source file uses the `.phont` extension and executes the same engine transactionally. PhonoScript is the language name; “phont” is not a separate executable or language. The software distinguishes a calculated negative result from a request that could not be formed or admitted. Missing dependencies therefore return a structured NOT EVALUATED record rather than false, zero, NaN, or an empty answer.

2 Installation and first project

The locally exercised Apple-silicon package is compiled `compiled/macos/PhonoScriptGUI.app`. It registers `.ottab` and `.phont` document types. Linux and Windows archives produced by the macOS cross-build gate are linked for their declared targets and inspected for binary format and path hygiene, but they are not native execution evidence. The matching release-workflow jobs remain the runtime gate on Linux and Windows. To build the shared application from source, install Rust 1.88 or newer and run:

```
cargo run --release -p convalgen --bin phonoscript-gui
```

The independent interpreter can be installed from source with:

```
cargo install --locked --path phonoscript --bin phonoscript
```

Native package commands are run from the same repository root on their matching operating systems:

```
./convalgen/source/scripts/package-macos.sh
./convalgen/source/scripts/package-linux.sh
powershell -ExecutionPolicy Bypass \
  -File .\convalgen\source\scripts\package-windows.ps1
```

PhonoScript GUI packages include the PhonoScript interpreter. Standalone language packages are maintained separately under `phonoscript/compiled/`. A package name records its operating system and architecture. The locally inspected macOS package does not constitute local execution evidence for the Windows or Linux packages. Its binaries are ad-hoc signed for local integrity, not Developer-ID signed or Apple-notarized. Public Gatekeeper distribution therefore remains an author-controlled release step.

Choose **File** → **New analysis**, name the project in the Inspector, and select the evaluator in the toolbar. Save with **File** → **Save As**. A project is one portable JSON document; it does not retain the path

from which it was opened, transient windows, export destinations, or cached evaluator results. Exact scalars and explicitly approximate scalars remain distinct in schema version 4. Unset violation cells are displayed as an em dash and are stored with a reserved non-mathematical marker. The engine refuses evaluation, learning, comparison, and export until the phonologist enters every count.

3 Workspace and project organization

The desktop follows a conventional scientific-application layout: a compact menu and toolbar, a resizable project navigator, a central editor, a contextual inspector, an optional command console, and a status bar. The palette is white, graphite, slate, and light gray with one muted steel-blue focus color. Text, symbols, line weight, and patterns carry analytical meaning independently of color.

The project navigator lists tableaux and analysis workspaces. Project actions add, duplicate, remove, rename, and reorder tableaux; copy a declared constraint register across a dataset; designate source and target tableaux; evaluate a project overview; and batch-export the current project. A project retains at least one tableau. Count labels use *tableau* for one item and *tableaux* for two or more.

The layout responds by reallocating space rather than uniformly shrinking content. At narrow widths, a workspace selector replaces the navigator and the inspector can collapse. Dense matrices keep independent horizontal and vertical scrollbars. Candidate rows are virtualized, columns are resizable, and row filtering changes only the working view rather than the candidate support. Long labels truncate with an ellipsis in bounded controls and expose their complete value through editing or hover text.

Preferences controls shortcut profile, panel visibility, compact or comfortable row density, metadata and legend visibility, and PNG export scale. Help summarizes the active data and calculation boundaries. About identifies Alexandre Menezes Barroso, alexandrebarroso.com, 2026, and the MIT License.

4 Tableau editing

The Tableau editor has two action rows. Candidate actions add, duplicate, move, and remove rows. Constraint actions add, move, tie, make strict, and remove columns. Moving a constraint also moves every corresponding violation cell. In OT, movement establishes a strict ranking by column order; **Tie left** places the selected constraint in the preceding stratum.

Click the input, a candidate identity or form, a constraint name, a rank or weight, or a phonologist-entered violation to edit it directly in place. MaxEnt observed counts and positive base masses are also input cells. Press **Delete** to clear the selected phonologist-entered violation. Derived costs, unnormalized masses, normalizers, and probabilities are read-only. Every ordinary or structured tableau's violation vector is supplied by the phonologist. PhonoScript GUI does not infer marks from a constraint name, descriptive definition, surface string, or typed phonological structure.

A row is a candidate for output. The guide uses *output* only for a candidate selected by the evaluator; editable rows are never labelled as outputs merely because they appear in a tableau.

4.1 Tie policies

Each tableau declares one winner-tie policy:

Retain all co-winners

preserves the evaluator's complete optimal set.

Choose first listed

resolves an evaluator tie by stable row order and records that policy.

Require a unique winner

returns no winner and an explicit unresolved-tie state when more than one native optimum exists.

The complete order remains available even when the displayed winner policy selects one co-winner. This preserves the distinction between equal winners and equal complete responses.

5 Evaluator mathematics**5.1 Optimality Theory**

OT compares candidates lexicographically by ranked strata. Constraints within one stratum are simultaneous. The result includes the native winner set, complete ordered tiers, and the first fatal constraint for each loser whenever that coordinate is unique. Strict OT has no scalar Harmony coordinate, so an OT row reports no Harmony value rather than a numerical zero.

5.2 Harmonic Grammar

HG minimizes the weighted sum $H(c) = \sum_i w_i v_i(c)$. Weights are stored as exact or explicitly approximate scalars; violations are nonnegative integers. A candidate cost remains an exact rational when every enabled weight is exact. A cost that contains an explicitly approximate weight crosses the declared numerical boundary. Persisted exact rationals do not become approximate in the project file, and an approximate result is not presented as an exact theorem.

5.3 Maximum Entropy

For candidate c , base mass $\rho(c) > 0$, energy $H(c)$, and temperature $T > 0$, PhonoScript GUI calculates

$$p(c \mid x) = \frac{\rho(c)e^{-H(c)/T}}{\sum_{d \in \text{Gen}(x)} \rho(d)e^{-H(d)/T}}.$$

Stable log-sum-exp normalization prevents ordinary overflow and underflow. Modal selection uses both energy and base mass. Exact Second-Order probability comparison uses symbolic finite exponential-polynomial masses where its declared algebraic conditions are satisfied; approximate and grid modes remain explicitly separate. Ordinary exponentials and normalized probabilities are approximate engine results even when the underlying weighted costs are exact.

5.4 Serial evaluation

Serial analyses declare GEN1 transitions, an identity candidate, constraints, and a step limit. The engine reports the path, operations, and stopping reason. It refuses missing candidate sets, cycles, ambiguous local winners, and a reached step limit instead of silently truncating the derivation.

The implemented evaluator domain is finite. PhonoScript GUI does not provide a general continuous-HG candidate space, continuous optimizer, KKT/persistence certificate, support-threshold engine, or contact/inversion solver. A grid-mode comparison checks declared finite sample points; it is not a certificate over a continuous domain.

6 Second-Order Tableau and Q-Calculus

Open **Second-Order Tableau** to register a source, target, query, answer sort, scope, transformation, transport, comparison mode, response domain, normalizer policy, scientific layers, and optional later consumer. The result is exactly one of:

- a preservation certificate;
- a complete discrepancy record; or
- an indexed refusal naming the failed formation or admission coordinate.

Queries include winner set, surface winner set, complete order, probability law, and candidate support. Terminal-result and complete-trajectory responses are distinct. Candidate renaming and explicitly mass-preserving fusion are typed transports. Shared and independent MaxEnt normalizers are not interchangeable declarations. Source and target are evaluated independently with their own tableau-level evaluator and temperature overrides when present, otherwise with the project defaults. These per-tableau settings therefore change the Second-Order responses; they are not display metadata.

Those five choices are the direct query families currently exposed by the engine's `QueryKind` type. They form a bounded executable interface; they do not exhaust the dissertation's inventory of Second-Order theorem families or every calculation developed there.

The Q-Calculus audit calculates the exact finite ranking space before and after a declared constraint clone. It separately reports support preservation and changes in ranking shares. The dynamic program calculates the image of the ranking set without enumerating every permutation when states can be shared. The registered calculation requires strict OT, retained co-winner sets, no project-level a-priori ranking relations, exactly aligned nonempty tableaux, 1–60 enabled constraints, and at most 63 candidates per tableau. Counts and reduced shares are arbitrary-precision exact values. Reaching the default 2,000,000-state search budget is reported as a structured refusal, not an approximate count, and ranking shares are not token probabilities by default.

7 PhonoScript

PhonoScript 3 is a deterministic UTF-8 domain-specific language. It provides Unicode identifiers and strings, exact numbers, lists, records, `null`, lexical functions, immutable and mutable bindings, bounded loops, conditionals, project mutation, evaluator calls, learning and inference, assertions, comparisons, and publication export. Stable lexer, parser, static analysis, and runtime diagnostics carry source spans. A script commits its project and queued file effects only after the complete program and resulting document validate.

```
project_title("Kager final-voicing fragment")
project_evaluator("OT")
tableau_select("source")
tableau_name("Dutch /bed/")
tableau_input("/bed/")
tableau_source_locator("Kager 1999, printed p. 16")
constraints_clear()
candidates_clear()
constraint_add("*VOICED-CODA", 1, "", 0)
constraint_add("IDENT-IO(voice)", 1, "", 1)
candidate_add("faithful", "[bed]", [1, 0])
candidate_add("devoiced", "[bet]", [0, 1])
assert_winners(["devoiced"])
```

Run an installed script with `phonoscript SCRIPT.phont`, or use **Analysis** → **Run PhonoScript script** in the application. A source-tree run uses:

```
cargo run --release -p phonoscript -- SCRIPT.phont
```

Use `--check` for parsing and static analysis without execution. The language also exposes typed segment features, morphology, prosody, tone, correspondence and derivation records. Declared finite GEN returns `complete`, `truncated`, or `refused`; completeness is always relative to the stated domain and resource contract. Importing a fully structured candidate still requires an explicit phonologist-supplied violation vector. Importing a generated candidate set likewise requires either one explicit vector for every candidate or an explicit matrix with one vector per candidate. No import derives marks from forms, segment features, prosodic structure, constraint names, or constraint definitions. The complete syntax, runtime, and engine contract is in `docs/PhonoScript-Language-Manual.pdf`.

PhonoScript 3 supports selective imports from local `.phont` modules. Only explicitly exported names are visible, imported bindings are immutable, and imported files are declaration-only. Project mutation, output, and file effects must be placed in an exported function and occur only when the entry script calls it. Saved module entries are checked and run under a canonical root equal to the entry file's containing directory; relative imports, cycles, symlink escapes, missing exports, and resource-limit failures retain the responsible file and source span in the diagnostic. An unsaved editor buffer with imports is refused until it has a stable file location. There is no remote source loader or package registry. Opening a base `.ottab` project supplies data, not executable declarations. The source editor keeps long lines unwrapped and exposes horizontal and vertical scrollbars only when needed; syntax colouring, diagnostic underlines, and cursor editing share the same parsed source.

8 Publication export

The Export menu writes the current tableau, Second-Order Tableau, plot, or all project tableaux as SVG, PNG, or PDF. SVG is the editable master. PNG scale and the inclusion of title, author, and legend are set in Preferences. The native Rust renderer carries the visual specification of the bundled `secondordertableau.sty` package into all three formats, crops every figure to its actual content, and never emits an intermediate `.tex` file.

An exported first-order OT tableau follows conventional semantics: constraints are ranked left to right, a heavier separator marks a stratum boundary, violations are stars, fatal violations carry `!`, and the optimal row carries a pointing hand plus redundant typographic emphasis. HG and MaxEnt use numeric violations, weights, costs, base masses, and probabilities. Color is restrained and never the sole carrier of a result.

The checked visual regression corpus exercises strict OT, HG, MaxEnt, serial records and refusals, Q-Calculus, plot families, and dense stress tableaux in SVG, PNG, and PDF. Its Second-Order scenes cover overlay preservation, delta-sidecar discrepancy, expanded paired refusal, and simultaneous terminal/trajectory lanes. The large-tableau case also checks semantic A3-landscape PDF tiling: page breaks occur only between complete candidate rows and constraint or metric columns, while each continuation page repeats the input/header band, relevant constraint labels, and candidate register. The number of pages is content-dependent rather than a public compatibility contract.

SVG output keeps editable text and semantic groups. PDF retains vector text and line work with embedded fonts. PNG dimensions follow the explicit numeric SVG canvas and declared scale; the SVG background also uses explicit numeric dimensions so native previewers do not reinterpret a percentage-sized background. All default output is cropped to content. A very large raster request is refused structurally while SVG and PDF remain available.

The exporter was inspected for long IPA, combining marks, bidirectional Arabic, winner and arrow glyphs, tied strata, exact rational values, refusal text, and dense matrices. Candidate rows are always labelled as candidates. The word *output* is reserved for a selected candidate or declared serial terminal result. Second-Order outcomes remain legible in monochrome through text, symbols, line geometry, and complete certificate or refusal records.

Live interface inspection covered compact, standard, wide, and portrait windows; the dissertation overview; large horizontally and vertically scrollable tableaux; PhonoScript diagnostics; Second-Order discrepancy; Preferences, Help, About, export, and save dialogs; plots; Q-Calculus; and structured refusal states. This establishes the inspected macOS states rather than pixel identity on every platform or display configuration.

8.1 Bundled tableau package

The repository includes `secondordertableau 1.0.0`, released 13 August 2026. A LaTeX user can place `secondordertableau.sty` beside a document or install it in a TeX search directory and load it with `\usepackage{secondordertableau}`. It supports pdfLaTeX, XeLaTeX, and LuaLaTeX, requires no shell escape, and writes no custom auxiliary data format.

The package typesets strict-OT, HG, and MaxEnt tableaux; semantic anchors; source-preserving Second-Order composition; typed queries and answer transport; mismatch and witness marks; maps and sidecars; and English or Brazilian Portuguese disclosure. It renders declared records. It does not generate candidates, calculate Harmony, normalize probabilities, select outputs, or prove a linguistic conclusion. Its package-owned tagged Figure does not certify a complete document as PDF/UA conforming.

The style package is distributed under the LaTeX Project Public License 1.3c or later. Maintenance status is *maintained*; the Current Maintainer is Alexandre Menezes Barroso. PhonoScript GUI’s native renderer uses the package as a visual specification but does not execute it.

8.2 Application icon

The Project PhonoScript wordmark is the all-lowercase word “phonoscript” in sky blue on a transparent field. The PhonoScript GUI mark is a single lowercase “p” in the same color and with the same transparent field. Both marks use Afacad Semibold, converted to deterministic vector outlines so neither file depends on an installed font. Afacad is copyright 2023 The Afacad Project Authors and is available under the SIL Open Font License 1.1. The wordmark source is `docs/assets/phonoscript-wordmark.svg`; the GUI source is `convalgen/source/assets/icon/phonoscript-gui-icon.svg`. PNG, ICNS, and ICO application assets are mechanical derivatives of the GUI source.

9 Keyboard and preferences

Action	Standard shortcut
Save	Cmd+S
Evaluate or run PhonoScript	Cmd+R
Add candidate	Cmd+Return
Add constraint	Shift+Cmd+Return
Duplicate candidate	Cmd+D
Clear selected violation	Delete
Move candidate	Option+Up / Option+Down

Move constraint	Option+Left / Option+Right
Command console	Shift+Cmd+P

Preferences provides standard, laptop-friendly, and disabled editing-shortcut profiles, workspace panel visibility, compact rows, and publication-export defaults. At narrow window widths a workspace selector replaces the navigator, so every principal screen remains reachable at the 640×480 minimum.

10 Validation and limits

The corpus test discovers every checked-in authored PhonoScript program, runs it transactionally, and checks its assertions. The reference-project test similarly enumerates the checked-in `.ottab` fixtures, validates each document, and evaluates its declared oracle boundary. The manuals do not freeze suite, script, fixture, or visual-scene totals that become stale whenever a validated asset is added.

The dissertation project contains 39 bounded transcription records: 32 have declared winner or modal oracles, and seven are intentionally neutral. Every record is anchored by an English/Portuguese-shared LaTeX label such as `fig:...` or `tab:...` in `source_locator`; names such as H.1 are navigation labels, not provenance keys. Its fidelity tag distinguishes a direct finite replay (`fidelity:exact`), a declared answer-preserving transformation (`fidelity:exact-transform`), a ledger without an oracle (`fidelity:neutral-ledger`), a bounded state (`fidelity:snapshot-only`), a non-serial flattened panel (`fidelity:flattened-panel`), or a source panel without the complete Second-Order result (`fidelity:source-panel-only`).

Source-exact finite conformance covers Kager’s two printed Dutch/English final-voicing panels, both panels of Pater’s weighted coda-devoicing Tableau (13), and all four printed Anttila–Cho competition counts across the three English subgrammars. The Tessier /*sku*l/ case preserves the printed support, weights, violations, and costs 11, 1, and 2; it corrects the source’s decimal for $\exp(-11)$ and supplies the omitted finite MaxEnt normalizer. The resulting approximate probabilities are engine-derived on that finite support, not a claim that Tessier printed a normalized law, a complete GEN, or learned weights. The authored PhonoScript and `.ottab` versions of the Goldwater–Johnson Tables 2–3 eleven-constraint ledger are instead exact printed-ledger transcriptions with the same structured refusal: fitted-probability replay is not formed because the learned weight vector is not printed.

Bounded reconstruction covers the printed rows a–b and spreading-first obstruction of McCarthy’s Rimi Tableau (17), the tagged dissertation records, the registered dissertation Second-Order contracts, and one finite MaxEnt fusion contract. The partial Prince–Smolensky Berber fixture is source-inspired: it tests the named ONS/HNUC mechanism without claiming source replication. The Kager basic-syllable, harmonic-bounding, and tie/order scripts and the Finnish-shaped Q assets are synthetic or derivative engine checks rather than complete source transcriptions.

The automated gates include focused unit, command-line, cross-interface, property, stress, corpus-discovery, reference-project, GUI, and editor checks. Where an independent native transcription exists, cross-interface tests compare it with both the authored script and the project fixture. The dissertation fixture is also checked for byte identity with the project distributed with PhonoScript GUI.

These checks are reproducible regression evidence. They do not prove every possible grammar, establish that a registered query is scientifically important, certify an approximate numerical center as exact, or convert finite empirical data into population evidence. They do not establish continuous-HG certificates, native Windows or Linux execution, or a notarized public macOS release. The complete language, fixture, and source-ceiling record is contained in `docs/PhonoScript-Language-Manual.pdf`.

Run the shared verification from the repository root:

```
cargo fmt --all -- --check
```

```
cargo clippy --workspace --all-targets --all-features -- -D warnings
cargo test --workspace --locked --all-targets
cargo run --release -p phonoscript --bin qcalc-bench
```

The fixed benchmark covers exact ranking-space counting, 50,000 finite-MaxEnt evaluations, and 20,000 typed Second-Order comparisons. Its encoded budgets are release gates on the executing machine, not universal hardware promises.

11 Security and contribution discipline

Treat an unfamiliar .phont or .ottab file as untrusted input. Review explicit save and export destinations before execution and remove confidential research data from a shared reproduction. A suspected vulnerability can be reported privately through alexandrebarroso.com.

An evaluator change requires a manually checkable finite case, an automated regression, a declared response type and exactness boundary, and a precise source locator when it reproduces published work. Missing candidates, weights, normalizers, transports, or observation bridges must not be replaced by invented values. The source-exact, bounded, derivative, and refusal labels in the PhonoScript manual must remain distinct.

12 License and attribution

PhonoScript GUI is free and open-source software under the MIT License.

Copyright © 2026 Alexandre Menezes Barroso.

alexandrebarroso.com